NAME: Ayaan Shaikh
Mobile no : 8055733707
Internship : Data Analytics using Python
Submission date: 21/05/2026

Task 61 : Understand relationships between variables in the dataset.

Objective

To explore and identify how different variables in a dataset are related to each other. This includes finding correlations, dependencies, and patterns between features, which helps in better decision-making, prediction, and understanding the overall structure of the data.

```
# Understand relationships between variables in the dataset.
import pandas as pd
import seaborn as sns
df=pd.read_csv('analysis_dataset.csv')
print(df.corr(numeric_only=True))
sns.heatmap(df.corr(numeric_only=True),annot=True,cmap='coolwarm')
```

```
              Value  Feature1  Feature2   Target     Price
Value      1.000000  0.185451  0.185451 -0.01846  0.185451
Feature1   0.185451  1.000000  1.000000  0.00000  1.000000
Feature2   0.185451  1.000000  1.000000  0.00000  1.000000
Target    -0.018460  0.000000  0.000000  1.00000  0.000000
Price      0.185451  1.000000  1.000000  0.00000  1.000000
```

You loaded a dataset using pandas and analyzed the relationships between its numerical variables by calculating the correlation matrix. Then, you used Seaborn to visualize these correlations in the form of a heatmap, where each cell represents the strength and direction of the relationship between two variables. By adding annotations and a color map, you made the visualization more informative and easy to interpret, helping to quickly identify strong positive or negative correlations within the dataset.

Task 62 : Use SQL for table and query analysis .

Objective

To use SQL for extracting, filtering, and analyzing data stored in databases. This helps in performing operations like selecting specific records, joining multiple tables, aggregating data, and generating meaningful insights efficiently from large datasets.

```
import sqlite3
conn=sqlite3.connect("shopping.db")
cursor=conn.cursor()
df=pd.read_sql('SELECT * FROM customer',conn)
print(df)
cursor.execute('SELECT AVG(age) FROM customer')
print(cursor.fetchone())
cursor.execute('SELECT name,COUNT("Harsh") FROM customer GROUP BY name')
print(cursor.fetchall())
cursor.execute('SELECT  * FROM customer WHERE age > 21 ')
print(cursor.fetchall())
```

```
    id    name  age
0  100   HARSH   22
1  200  Pritam   21
2  300    Amit   24
3  410   Sneha   23
4  500   Rahul   20
(22.0,)
[('Amit', 1), ('HARSH', 1), ('Pritam', 1), ('Rahul', 1), ('Sneha', 1)]
[(100, 'HARSH', 22), (300, 'Amit', 24), (410, 'Sneha', 23)]
```

I connected to a SQLite database named *shopping.db* using the sqlite3 library and created a cursor to execute SQL queries. Then, I retrieved the entire *customer* table into a pandas DataFrame and displayed it for analysis.

After that, I performed different SQL operations such as calculating the average age of customers, counting occurrences of a specific name while grouping data by name, and

filtering records to display only those customers whose age is greater than 21. These steps helped me to explore, summarize, and analyze the data stored in the database.

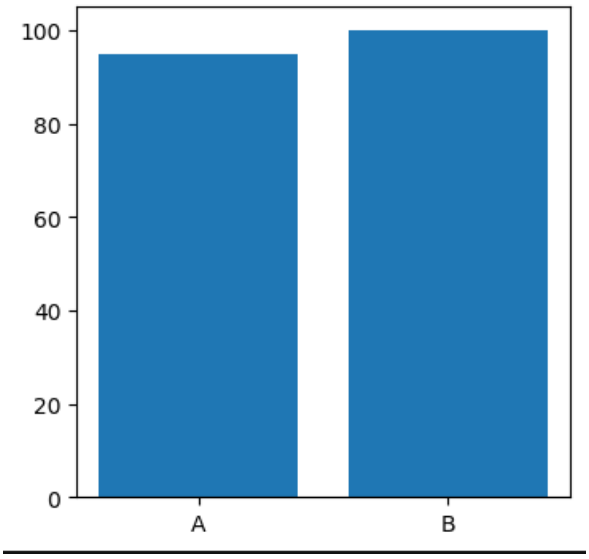
Task 63 : Create different types of graphs and charts.

Objective:

To visually represent data using various types of charts such as bar charts, line graphs, scatter plots, and histograms. This helps in simplifying complex data, identifying patterns, and communicating insights clearly to others .

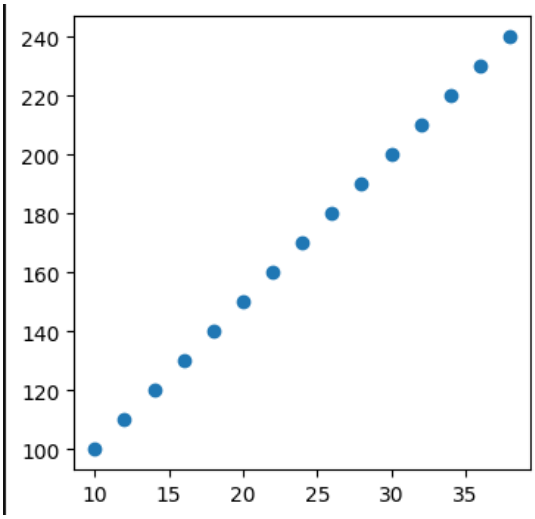
```
# Create different types of graphs and charts
import matplotlib.pyplot as plt
df=pd.read_csv('analysis_dataset.csv')
plt.figure(figsize=(4,4))
#plt.scatter(df['Feature1'],df['Feature2'])
plt.bar(df['Category'],df['Value'])
plt.show()
```

Output :



```
plt.figure(figsize=(4,4))
plt.scatter(df['Feature1'],df['Feature2'])
plt.show()
```

Output :



You loaded a dataset from a CSV file using pandas and created visualizations using Matplotlib. First, you generated a bar chart to represent the relationship between categories and their corresponding values, adjusting the figure size for better display.

Then, you created a scatter plot to analyze the relationship between two numerical features (*Feature1* and *Feature2*), which helps in identifying patterns, trends, or correlations between them.

These visualizations allowed you to better understand and interpret the data through graphical representation.

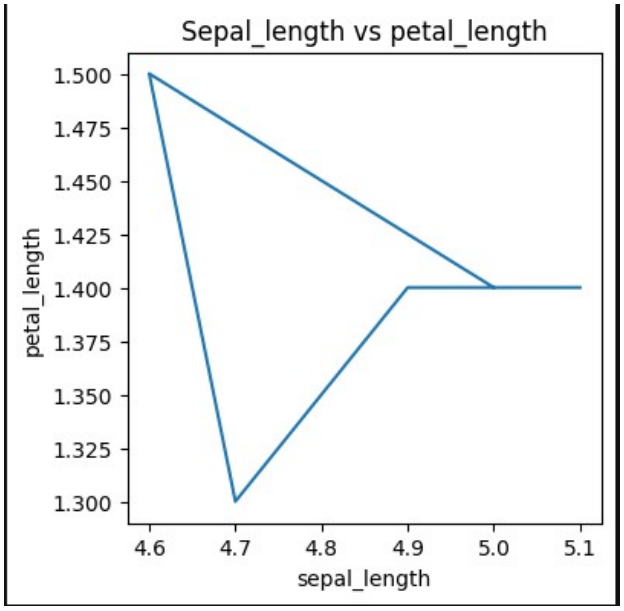
Task 64 : Create basic plots using Matplotlib .

Objective

To learn how to create fundamental visualizations using Matplotlib in Python, such as line plots, bar charts, and histograms. This builds a strong foundation in data visualization and helps in understanding how data behaves.

```python
# 64. Create basic plots using Matplotlib
df=pd.read_csv('iris.csv')
plt.figure(figsize=(4,4))
plt.plot(df['sepal_length'].head(5),df['petal_length'].head(5))
plt.title("Sepal_length vs petal_length")
plt.xlabel('sepal_length')
plt.ylabel('petal_length')
plt.show()
```

Output :



I loaded the *iris* dataset using pandas and created a line plot using Matplotlib to visualize the relationship between sepal length and petal length. I selected the first five records from both columns to keep the visualization simple and clear.Then, I customized the plot by adding a title and labeling the x-axis and y-axis for better understanding.

Finally, I displayed the graph, which helps in observing how petal length changes with respect to sepal length for the selected data points.

Task 65: Create attractive and consistent visuals using Seaborn .

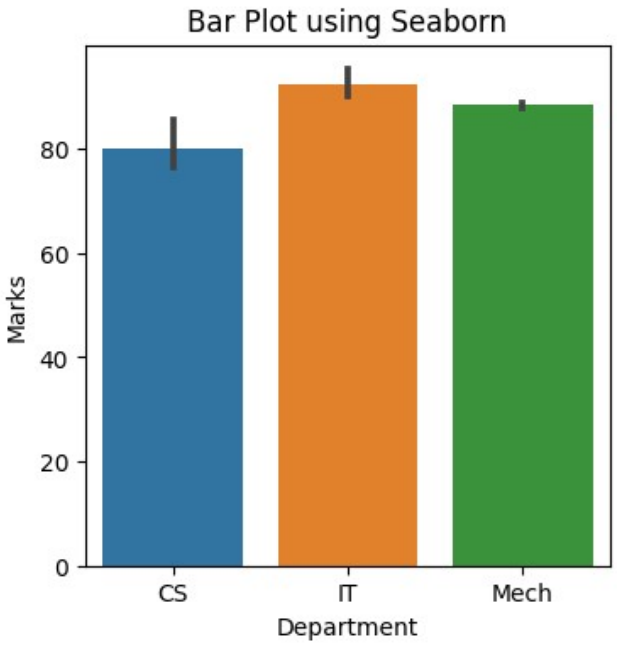
Objective

To create attractive, clear, and consistent data visualizations using Seaborn for better understanding and presentation of data.

```python
import seaborn as sns
import seaborn as sns
import matplotlib.pyplot as plt
df=pd.read_csv("students.csv")
x = df['Department']
y = df['Marks']
plt.figure(figsize=(4,4))
sns.barplot(x=x, y=y)

plt.title("Bar Plot using Seaborn")
plt.show()
```

Output :

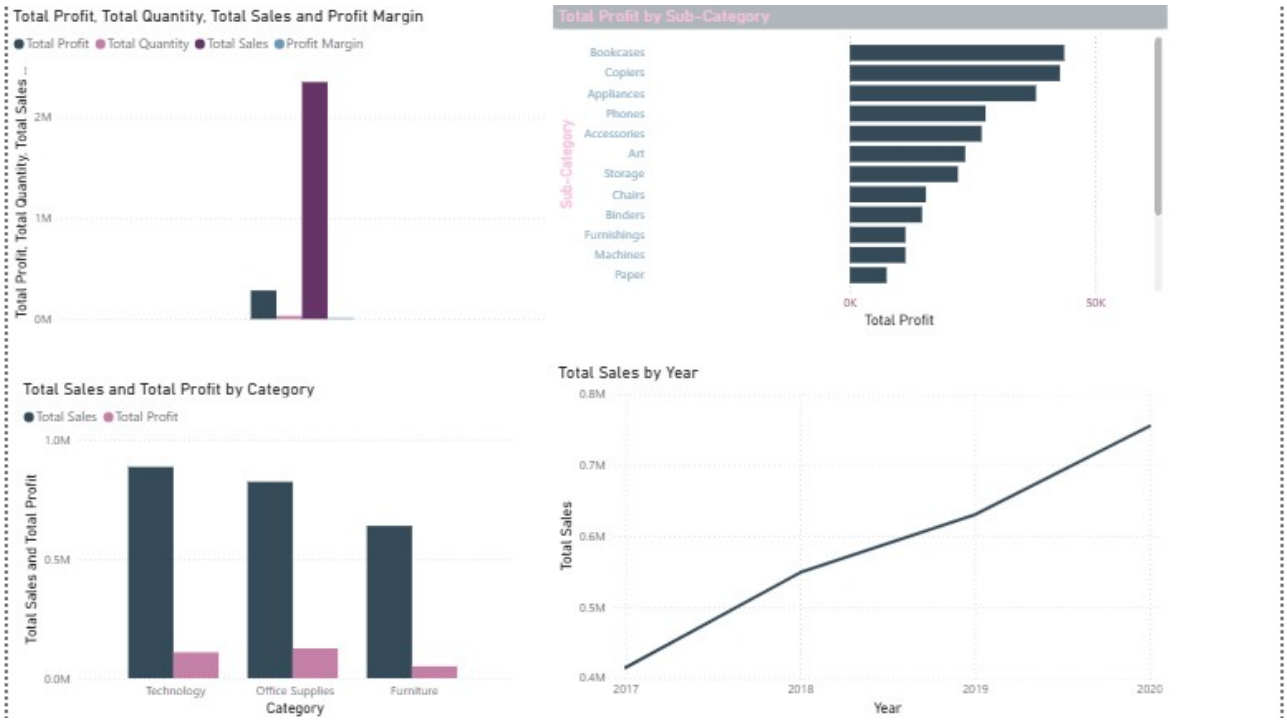


In this task, I used **Seaborn and Matplotlib in Python** to create a bar chart for visualizing student data. I first imported the required libraries and loaded the dataset (students.csv) using Pandas. Then, I selected **Department** as the categorical variable (x-axis) and **Marks** as the numerical variable (y-axis). I created a figure with a fixed size and used **Seaborn's barplot** to display the relationship between departments and their corresponding marks. Finally, I added a title to make the chart more understandable and displayed the visualization. This helped in comparing student performance across different departments in a clear and visually appealing way.

Task 66 : Use Tableau or Power BI for data visualization.

Objective

To use tools like Tableau or Power BI to create clear, interactive, and insightful data visualizations.



In this dashboard, I analyzed the sales dataset by creating multiple visualizations to understand business performance. I displayed key metrics like **Total Sales, Total Profit, Total Quantity, and Profit Margin** to get an overall summary.

I used a **bar chart to show total profit by sub-category**, which helps identify the most and least profitable products. Another chart compares **total sales and profit by category**, giving insight into which category performs best.

Additionally, I created a **line chart showing total sales over the years (2017–2020)** to analyze growth trends.

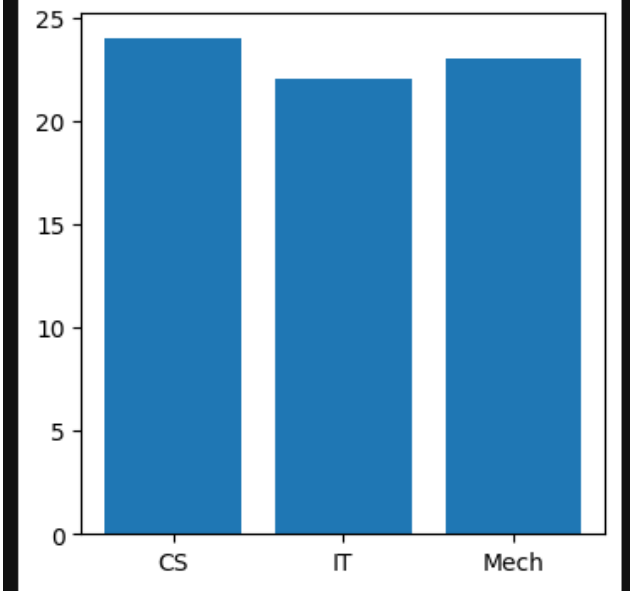
Overall, this dashboard helps in identifying sales patterns, profitable segments, and business performance over time, making it easier to support decision-making.

Task 67 : Create Bar Charts and Line Graphs .

Objective

To create bar charts and line graphs that help in comparing data across categories and identifying trends over time in a clear, structured, and visually appealing manner.

```
plt.figure(figsize=(4,4))
plt.bar(df['Department'],df['Age'])
plt.show()
```



In this task, I used **Matplotlib** to create a simple bar chart for data visualization. I set the figure size to make the chart compact and clear.

Then, I plotted a **bar graph** using the **Department** column on the x-axis and **Age** on the y-axis, which helps compare the age distribution across different departments.

Finally, I displayed the chart using plt.show(). This visualization makes it easier to understand how age varies among different departments in the dataset.

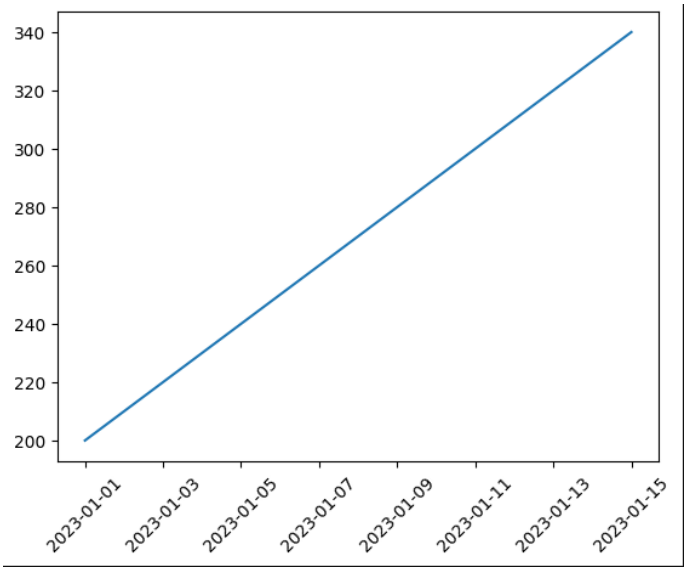
Task 68 : Create Data Timeline .

Objective

To create a data timeline that visualizes how data changes over a period of time, helping to identify trends, patterns, and growth effectively.

```
df=pd.read_csv("analysis_dataset.csv")
df['Date']=pd.to_datetime(df['Date'])
plt.plot(df['Date'],df['Price'])
plt.xticks(rotation=45)
plt.show()
```

Output :



In this task, I created a data timeline visualization using Python. I first loaded the dataset (analysis_dataset.csv) using Pandas and converted the Date column into a proper datetime format to enable time-based analysis.

Then, I used Matplotlib to plot a line graph with Date on the x-axis and Price on the y-axis, which shows how the price changes over time. I also rotated the x-axis labels for better readability. Finally, I displayed the chart using plt.show(). This visualization helps in identifying trends, patterns, and fluctuations in price over a period of time.

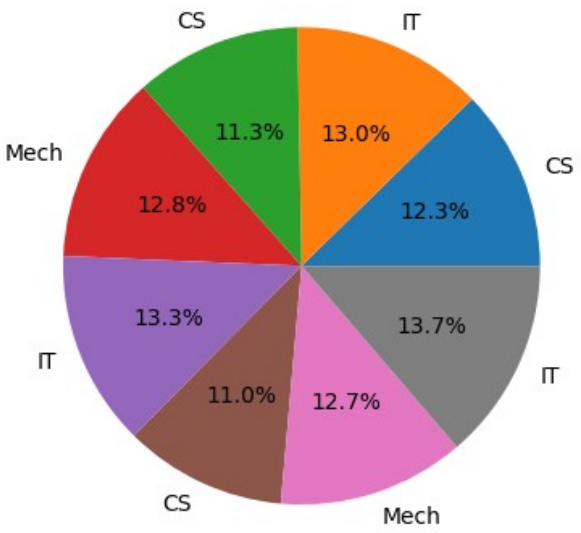
Task 69 : Use Pie Chart to show distribution .

Objective

To use a pie chart to represent the distribution of data across different categories in a clear and visually appealing way.

```
df=pd.read_csv("students.csv")
labels=df['Department']
plt.pie(df['Marks'],labels=labels,autopct='%1.1f%%')
plt.show()
```

Output :



In this task, I created a pie chart using Matplotlib to visualize the distribution of student marks across different departments. I first loaded the dataset (students.csv) using Pandas and selected the Department column as labels and Marks as values.

Then, I used plt.pie() to represent how marks are distributed among departments, and added percentage values using autopct for better understanding. Finally, I displayed the chart using plt.show(). This visualization helps in easily identifying the proportion of marks contributed by each department.

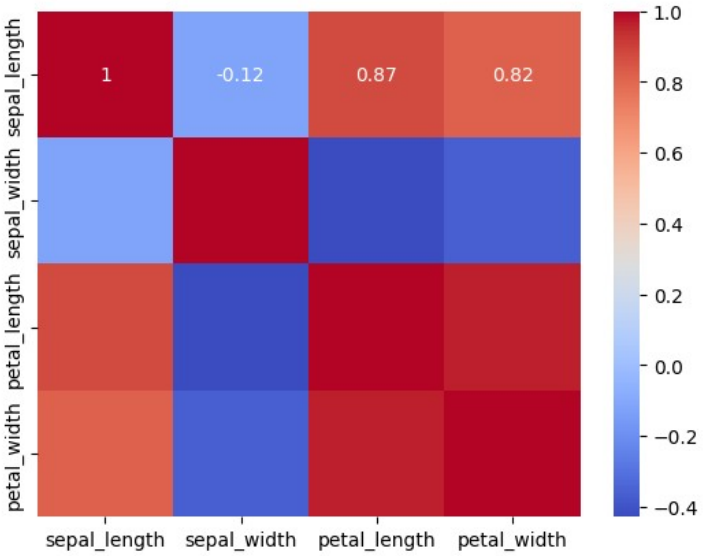
Task 70 : Create Heatmap .

Objective

To create a heatmap for visualizing relationships, patterns, and correlations between variables in a dataset using color intensity.

```
# 70. Create Heatmap
df=pd.read_csv("iris.csv")
sns.heatmap(df.corr(numeric_only=True),annot=True,cmap='coolwarm')
plt.show()
```

Output :



In this task, I created a heatmap visualization using Seaborn to analyze relationships within the dataset. I first loaded the iris dataset using Pandas, and then calculated the correlation matrix for all numerical columns using df.corr(numeric_only=True). After that, I used Seaborn's heatmap to display these correlations, where different colors represent the strength and direction of relationships between variables. I also enabled annotations (annot=True) to show the correlation values inside each cell and applied the 'coolwarm' color map for better visual clarity. Finally, I displayed the heatmap using plt.show(). This visualization helps in easily identifying strong positive or negative correlations between features in the dataset.